

- 6.1 进化算法的产生与发展
- 6.2 遗传算法及其应用
- 6.3 粒子群优化算法及其应用
- 6.4 蚁群算法及其应用(自学)

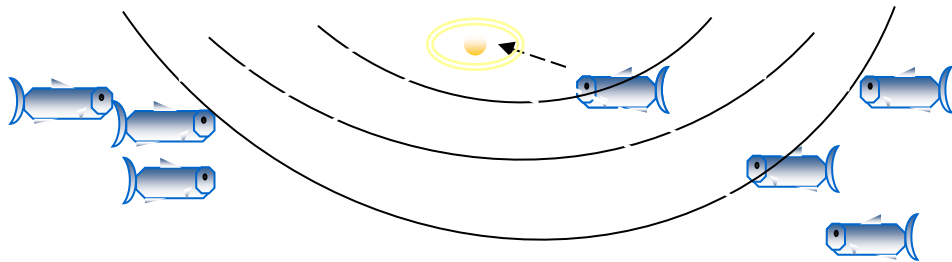
- 掌握**遗传算法**的基本概念和基本方法，了解一些遗传算法的改进算法，了解遗传算法的应用实例；
- 掌握**粒子群优化算法**等群智能算法的原理、步骤及应用，理解群智能算法与遗传算法的区别。



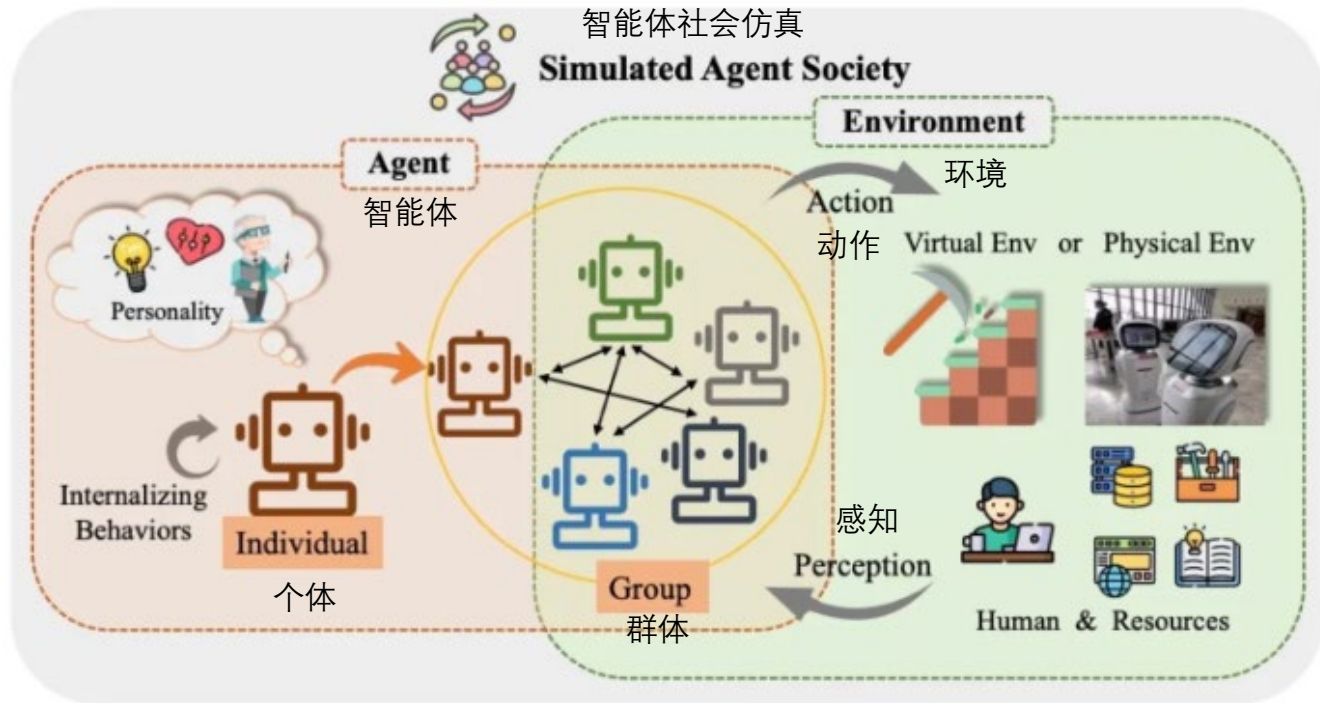
群体智能的起源：自然界群居生物的智慧模拟，1989年由赫拉多（Gerardo）等提出



群体智能(Swarm Intelligence, SI): 由简单个体组成的群落与环境以及个体之间的互动行为。

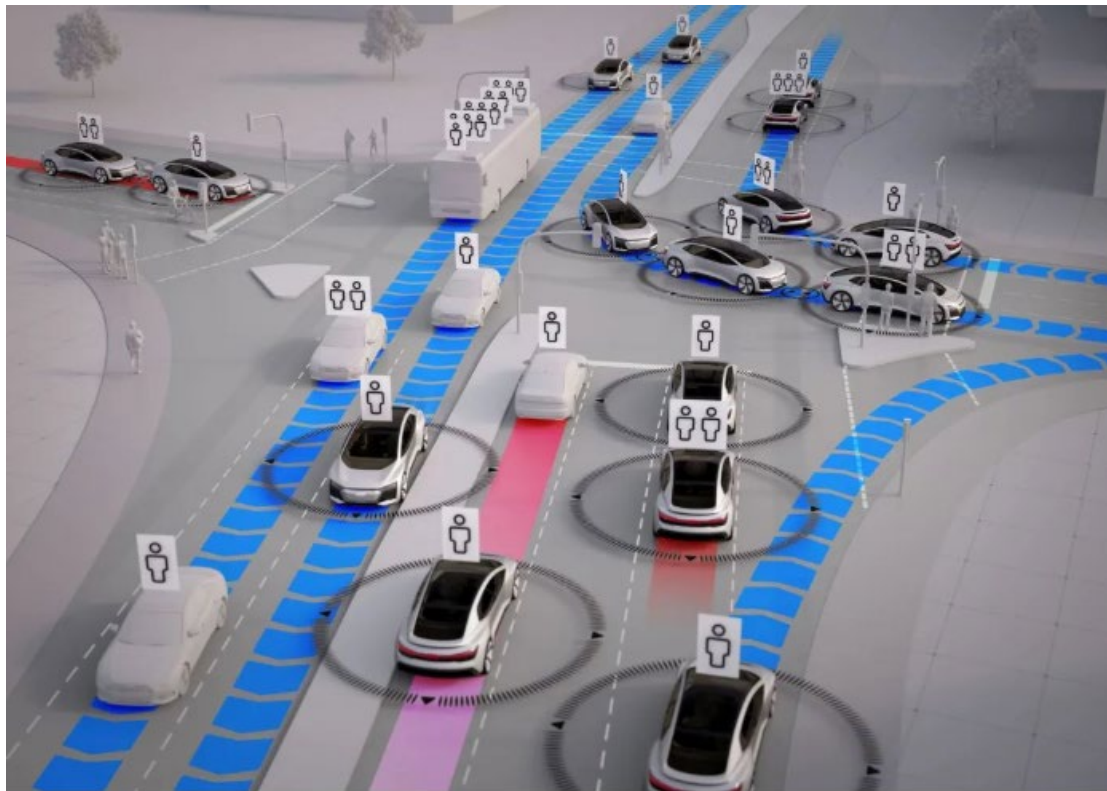


群体智能的核心在于分布式决策和协作，它通过算法让众多小型智能单元协同工作，共同解决复杂问题。





蜂群协同系统（OFFSET, **OFFensive Swarm-Enabled Tactics**）：由小型无人机系统（UASs）或小型无人地面车辆系统（UGSs）组成的“蜂群”，可在复杂的环境中通过分布式自主协同完成多种任务。



群体智能支撑的路径规划系统：奥迪的群体智能“Car-to-X”(车到万物)理念

■ **群智能算法 (Swarm Intelligence Algorithms)** : 受生物群体智能启发的算法。

■ 群智能算法包括: **粒子群优化算法 (Particle Swarm Optimization, PSO)**、**蚁群优化算法 (Ant Colony Optimization, ACO)**、**人工蜂群算法 (Artificial Bee Colony, ABC)**、**鱼群算法**、**细菌群算法**.....



蚁群搭桥



鸟群觅食



蜂群筑巢

- 6.3.1 粒子群优化算法的基本原理
- 6.3.2 粒子群优化算法的参数分析

掌握粒子群优化算法等群智能算法的原理、步骤及应用，理解群智能算法与遗传算法的区别

- ◆ 重点：粒子群算法原理（位置和速度的更新公式、适应度函数）及算法流程、算法参数分析
- ◆ 难点：粒子群算法应用（粒子的位置和速度的表示及更新等）



簇拥的鸟群

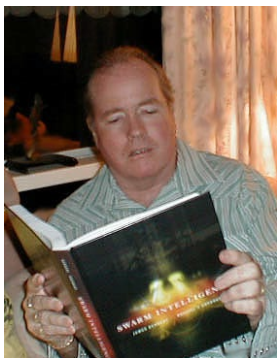
分离，避免拥挤临近单位

对齐，顺着周边单位的平均飞行方向

凝聚，保持周边单位的平均位置



粒子群优化算法（**Particle Swarm Optimization**, 简称**PSO**）：又称**微粒群算法**，由美国普渡大学的Kennedy和Eberhart于**1995年**提出，是一种基于群体智能的优化算法，灵感来源于对**鸟群觅食行为**的研究。



James
Kennedy



Russell
Eberhart



PSO是模拟鸟群飞行觅食行为的一种随机优化算法



» 一群鸟在随机搜寻食物，找到食物的最优策略是什么呢？

» 最简单有效的就是**搜寻目前离食物最近的鸟的周围区域**，**根据自己飞行的经验判断食物的所在**。

» Kennedy和Eberhart的初衷是研究鸟的觅食行为，建立一个模型来**模拟鸟群寻找食源**的现象。

» Kennedy和Eberhart在实验中发现，这个模型有很强的优化能力：**鸟之间通过集体的协作使群体达到最优目的**。



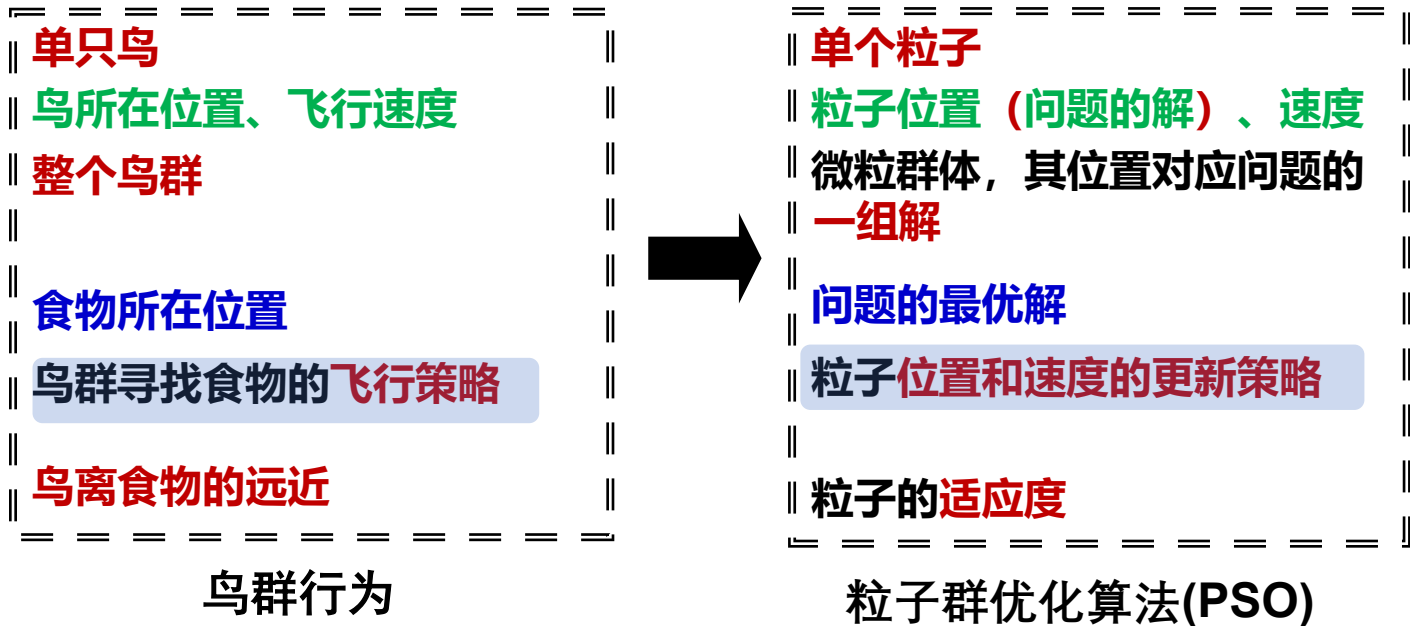
PSO是模拟鸟群飞行觅食行为的一种随机优化算法



怎么描述鸟群觅食行为模型？
怎么求解优化问题？



将问题的搜索空间类比于鸟类的觅食空间，将每一只鸟抽象为一个无质量无体积的**粒子 (Particle)**，用于表征每个候选解，优化寻求最优解等同于鸟群寻找食物。

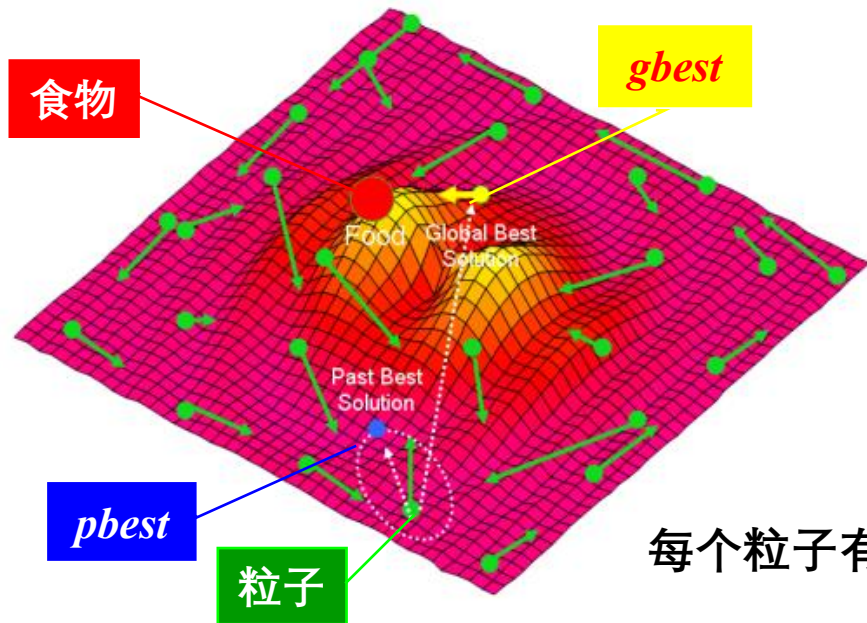


鸟群怎样尽快找到食物？

一个有效的方法就是**搜寻目前离食物最近同伴的位置**，**根据自己飞行的经验判断食物的所在**。

- » 每个粒子有一个**位置**和一个**速度**。粒子就是通过**自己的飞行经验和同伴中最好的飞行经验**来决定下一步的运动。
- » 粒子通过跟踪两个“极值”来更新自己。第一个就是**粒子本身目前所找到的最优解**，即个体历史最优***pbest***；另一个是**整个群体目前的最优解**，即群体最优个体***gbest***。
- » PSO为每个粒子制定了类似于鸟类运动的简单行为规则，从而使整个微粒群的运动表现出与鸟类觅食类似的特性，用于求解复杂优化问题。

粒子通过跟踪两个“极值”来更新自己。第一个就是粒子本身目前所找到的最优解 (Past Best Solution), 即个体历史最优 *pbest*; 另一个是整个群体目前的最优解 (Global Best Solution), 即群体最优个体 *gbest*。



每个粒子有一个位置和速度

假设搜索空间为 n 维，群体（Swarm）中的粒子数为 m 。

在第 t 次迭代中，群体中第 i 个粒子（ $i \in m$ ）的位置 x_i （所求解问题的一个解）

表示为： $x_i(t) = (x_{i1}, x_{i2}, \dots, x_{in})$

第 i 个粒子在飞行中所经过的最佳位置 p_i 为：

$$p_i(t) = (p_{i1}, p_{i2}, \dots, p_{in})$$

群体中所有粒子的 $p_i(t)$ 中最佳位置 p_g 为：

$$p_g(t) = (p_{g1}, p_{g2}, \dots, p_{gn})$$

第 i 个粒子的速度 v_i 为： $v_i(t) = (v_{i1}, v_{i2}, \dots, v_{in})$

所有的粒子都由一个适应度函数确定适应值（对应目标函数的值）以判断目前的位置好坏

速度决定粒子移动的方向和大小。粒子速度根据自身的飞行经验 $pbest$ （个体最优解）以及同伴（群体）的飞行经验 $gbest$ （群体最优解）进行动态调整。

粒子 i 的第 j 维的**速度和位置更新公式**:

惯性项

认知项

社会项

$$v_{ij}(t+1) = v_{ij}(t) + \boxed{c_1 r_1 (p_{ij}(t) - x_{ij}(t))} + \boxed{c_2 r_2 (p_{gj}(t) - x_{ij}(t))}$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

粒子经历过的最优位置 **pbest** : $p_i(t)$
群体经历过的最优位置 **gbest** : $p_g(t)$

粒子速度更新公式包括三项:

- ◆ **惯性项**: 指引粒子继续飞行的先前粒子速度。
- ◆ **认知项**: 粒子重新返回自身所经过的最好位置的趋势, 即粒子**自身经验**的影响
- ◆ **社会项**: 粒子被群体当前最好位置吸引的趋势, 即**群体经验**的影响

在三部分的共同作用下, 粒子根据**自身历史经验**并利用**群体全局信息**, 不断调整自己的位置, 以期找到**最优解**。

粒子 i 的第 j 维的**速度和位置更新公式**:

$$\begin{aligned}
 v_{ij}(t+1) &= \overset{\text{惯性项}}{v_{ij}(t)} + \overset{\text{认知项}}{c_1 r_1 (p_{ij}(t) - x_{ij}(t))} + \overset{\text{社会项}}{c_2 r_2 (p_{gj}(t) - x_{ij}(t))} \\
 x_{ij}(t+1) &= x_{ij}(t) + v_{ij}(t+1)
 \end{aligned}$$

粒子经历过的最优位置 **pbest** : $p_i(t)$
 群体经历过的最优位置 **gbest** : $p_g(t)$

其中 r_1 和 r_2 为 $[0,1]$ 间的随机数, 以增加搜索随机性; c_1 和 c_2 为**学习因子 (加速系数)**, 调节学习最大步长, 用来控制 $p_i(t)$ 和 $p_g(t)$ 对粒子飞行方向的影响。

若粒子 i 的位置的第 j 维超过范围 $[X_{\min}, X_{\max}]$, 则取边界值。

若粒子 i 的速度的第 j 维超过范围 $[V_{\min}, V_{\max}]$, 则取边界值。

位置限制: 自变量的取值范围, 对于无约束问题此处可以省略

惯性项

学习因子

认知项

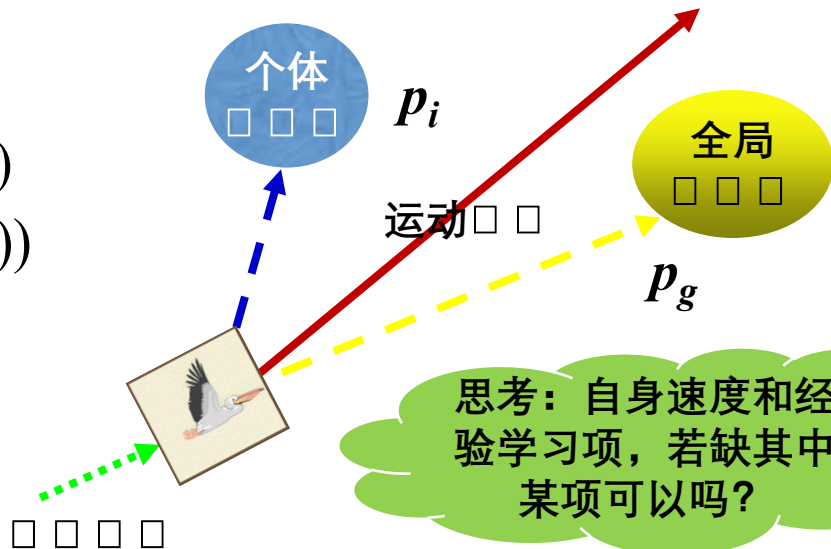
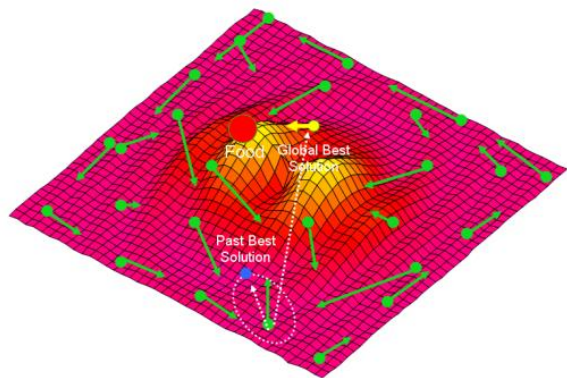
社会项

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_1 (p_{ij}(t) - x_{ij}(t)) + c_2 r_2 (p_{gj}(t) - x_{ij}(t))$$

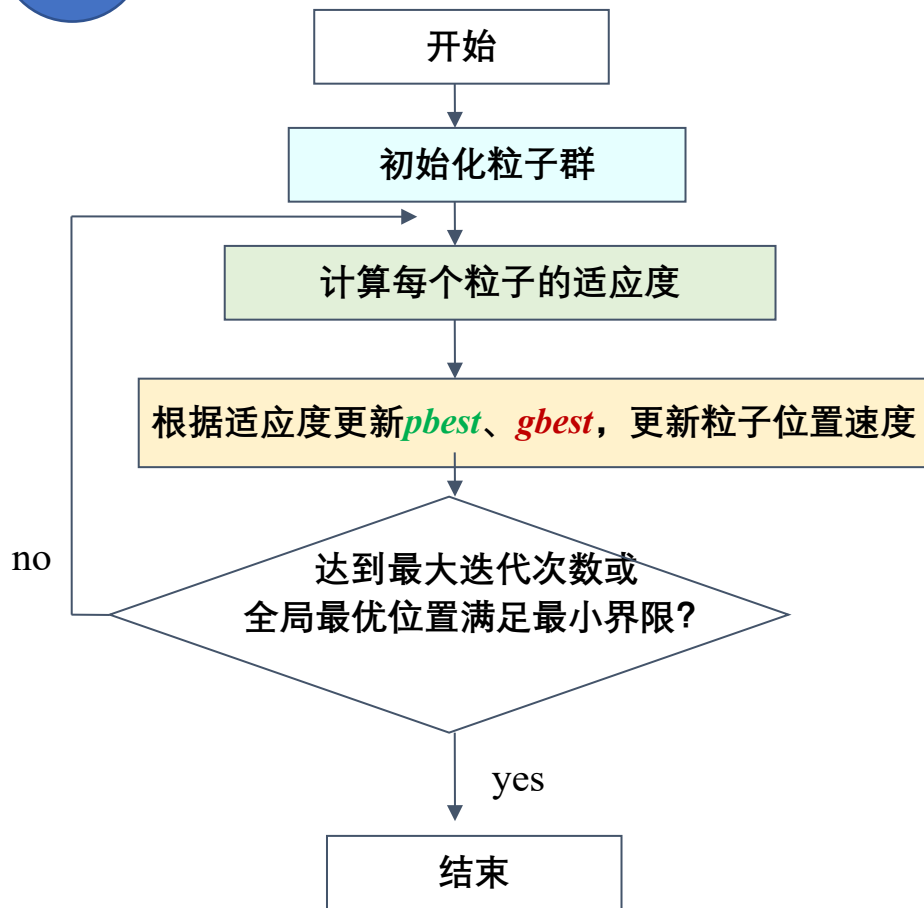
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

$$x_i(t) = (x_{i1}(t), x_{i2}(t), \dots, x_{in}(t))$$

$$v_i(t) = (v_{i1}(t), v_{i2}(t), \dots, v_{in}(t))$$



粒子经历过的最优位置 **pbest** : $p_i(t)$
 群体经历过的最优位置 **gbest** : $p_g(t)$



$$\min f(x_1, x_2) = x_1^2 + x_2^2$$

$$-10 \leq x_j \leq 10 \quad j = 1, 2$$

每个粒子有一个位置和一个速度：

$$x_i(t) = \{x_{i1}, x_{i2}, \dots, x_{in}\}$$

$$v_i(t) = \{v_{i1}, v_{i2}, \dots, v_{in}\}$$

粒子经历过的最优位置 *pbest* : $p_i(t)$
 群体经历过的最优位置 *gbest* : $p_g(t)$

$$v_{ij}(t+1) = v_{ij}(t) + \left[c_1 r_1 (p_{ij}(t) - x_{ij}(t)) \right] + \left[c_2 r_2 (p_{gj}(t) - x_{ij}(t)) \right]$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

种群规模很小：计算快，但种群多样性差，陷入局优的可能性很大

种群规模大：探索更充分，不易陷入局部最优，但计算成本高

当群体数目增长至一定水平时，再增长将不再有显著的作用

PSO对种群规模要求不高，一般取20~40就可以达到很好的求解效果。通常简单问题，10-20个粒子可能足够；对于复杂、高维问题，可能需要100个或更多。



惯性权重 w 控制着前一速度对当前速度的影响，用于平衡算法的全局探索和局部开发能力

惯性项（速度冲量） 导致粒子按照先前速度方向继续移动。Yuhui Shi^[1] (1998) 提出一个**惯性权重** w 来控制先前粒子速度的影响。

惯性权重

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

[1] Y. Shi, R. Eberhart. "A modified particle swarm optimizer," Proceedings of IEEE World Congress on Computational Intelligence, Anchorage, AK, 1998, pp. 69-73.

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

通过调节 w 值，可以控制PSO的**全局探索**和**局部开发能力**：

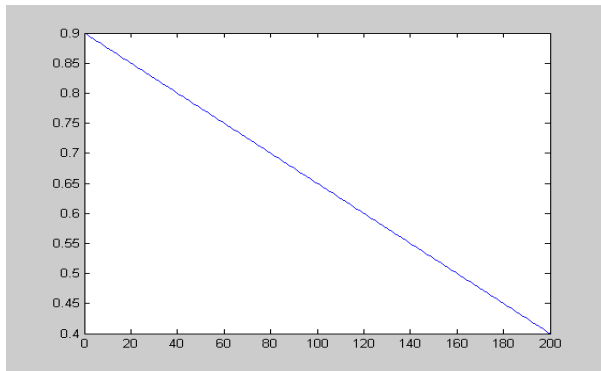
- $w \geq 1$ ：粒子速度随迭代次数的增加而增加，粒子发散。
- $0 < w < 1$ ：粒子**减速**，**算法的收敛性**依靠**加速系数** c_1 和 c_2 。
- $w < 0$ ：粒子**速度**随迭代次数的增加而减小，最后**趋近0**，**算法收敛**。

实验表明， $w=0.7298$ 和 $c_1 = c_2 = 1.497$ 时算法具有较好的收敛性能。

自适应或动态惯性权重值：

- Eberhart和Shi（1998）： w 的**线性递减策略**

$$w = w_{\max} - (w_{\max} - w_{\min}) * \frac{t}{t_{\max}}$$

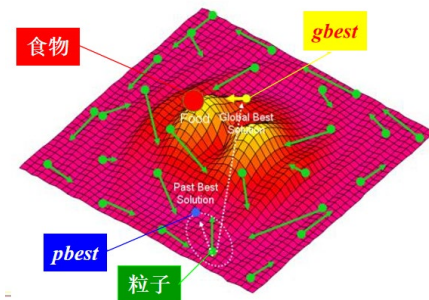


PSO学习因子

个体学习因子

群体学习因子

$$v_{ij}(t+1) = wv_{ij}(t) + c_1 r_1 (p_{ij}(t) - x_{ij}(t)) + c_2 r_2 (p_{gj}(t) - x_{ij}(t))$$
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$



加速系数(学习因子) c_1 和 c_2 ：控制 p_i 和 p_g 对微粒飞行方向的影响

粒子经历过的最优位置 $pbest$ ： $p_i(t)$
群体经历过的最优位置 $gbest$ ： $p_g(t)$

- $c_1 > 0, c_2 = 0$ ： **PSO认知模型(局部型PSO)**，没有信息的社会共享，导致算法收敛速度缓慢；
- $c_1 = 0, c_2 > 0$ ： **PSO社会模型(全局型PSO)**，迅速丧失群体多样性，易陷入局部最优而无法跳出；
- $c_1 > 0, c_2 > 0$ ： **PSO全模型**，易保持收敛速度和搜索效果的均衡，推荐取值范围： $[0, 4]$ ；典型取值： $c_1=c_2=2$ ；
- $c_1 > c_2$ ： 更强调自我学习，比较适合于单峰优化问题；
- $c_2 > c_1$ ： 更强调社会学习，比较适合于多峰优化问题。

最大速度 $V_{\max}$

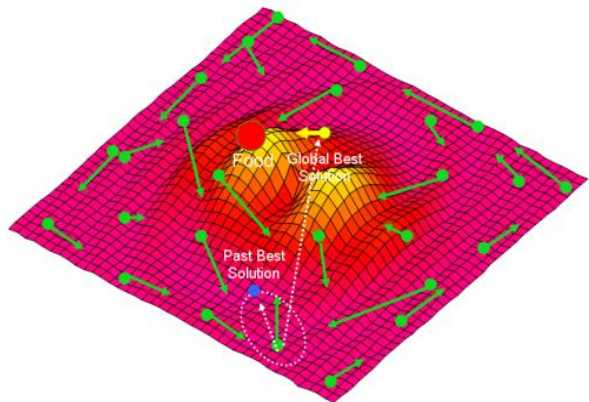
作用：限制粒子在每一维上速度的最大绝对值，可以维护算法的探索能力与开发能力的平衡，决定了当前位置与最好位置之间的区域的分辨率（或精度）

$V_{\max}$ 较大时，全局探索能力增强，但粒子容易飞过最优解，难以收敛

$V_{\max}$ 较小时，局部开发能力增强，容易陷入局部最优，收敛速度慢

$V_{\max}$ 通常与搜索空间的宽度相关，一般设为每维变量变化范围的10%~20%。

由于有惯性权重 w ，有时将 $V_{\max}$ 设为一个很大的值，而主要依靠 w 来控制探索。



智能计算

- 模糊逻辑
- 遗传算法
- 模拟退火算法
- 人工神经网络
- DNA计算
- 禁忌搜索算法
- 免疫算法
- 膜计算
- 量子计算
-

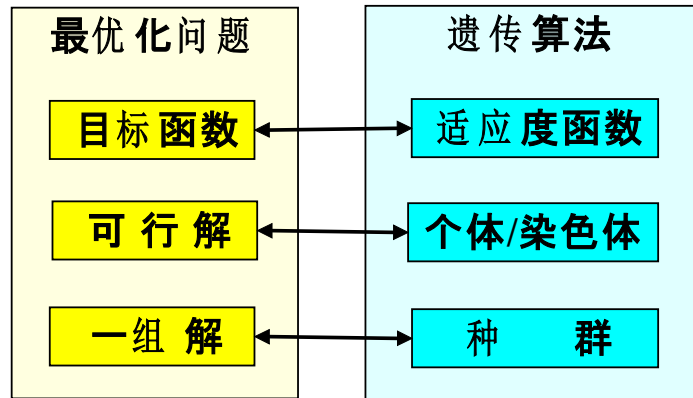
群
智
能
算
法

- 粒子群算法
- 蚁群算法
- 蜂群算法
- 鱼群算法
- 细菌群算法
-



相同点

- (1) 都属于仿生算法（受自然界启发的**元启发式**优化算法）
- (2) 都属于全局优化方法
- (3) 都属于随机搜索算法
- (4) 都隐含并行性
- (5) 根据个体的适应度进行搜索
- (6) 对高维复杂问题，会**早熟收敛**（陷入**局部最优解**）



鸟群觅食	粒子群算法
鸟	粒子
森林	求解空间
食物的量	目标函数值（适应值）
每只鸟所处的位置	空间中的一个解（粒子位置）
食物量最多的位置	全局最优解

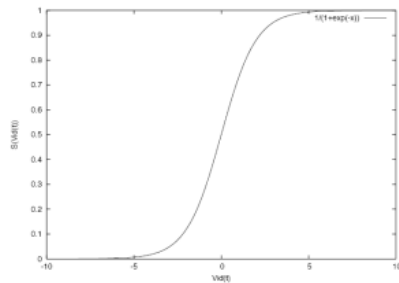
- ◆ 遗传算法强调“**适者生存**”，不好的个体在竞争中被淘汰
- ◆ PSO强调“**协同合作**”，不好的个体通过社会学习向好的方向转变
- 遗传算法中最好的个体通过产生更多的后代来**传播基因**
- PSO中的最好个体通过吸引其它个体向它靠近来**施加影响**
- 遗传算法的选择概率只与上一代群体相关，而**与历史无关**，群体的信息变化过程是一个Markov链过程
- PSO中的个体(粒子)除了有位置和速度外，还有着过去的**历史信息** (*pBest*、*gBest*)

基本粒子群算法 (PSO) 没有交叉和变异操作，粒子只是通过内部速度进行更新，因此原理简单、参数较少、实现容易、收敛速度快。

- ◆ 1997, Kennedy和Eberhart^[1]首次将实数PSO扩展为二进制PSO。
- ◆ 微粒位置为二进制向量, 微粒速度仍为浮点向量;
- ◆ 微粒速度被逻辑函数 $\text{sigmoid}(v)$ 转化为判断位置项选择0还是1的概率

$$x_{ij}(t+1) = \begin{cases} 0, & \text{if } \text{sigmoid}(v_{ij}(t+1)) < \text{rand}() \\ 1, & \text{otherwise} \end{cases}$$

$$\text{sigmoid}(v_{ij}) = \frac{1}{1 + e^{-v_{ij}}}$$



- 基本PSO是用来解决连续问题优化的, **二进制PSO**用来解决离散优化问题。
- **粒子的位置只有(0,1)两种状态**。速度值越大, 则粒子位置取1的可能性越大, 反之越小。

[1] J. Kennedy, R. C. Eberhart. "A discrete binary version of the particle swarm algorithm," Proceedings of International Conference on System, Man, and Cybernetics, 1997, Orlando, FL, pp. 4104-4109.

二进制PSO更新公式如下：

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = \begin{cases} 0, & \text{if } \text{sigmoid}(v_{ij}(t+1)) < \text{rand}() \\ 1, & \text{otherwise} \end{cases}$$

其中 $\text{sigmoid}(v_{ij}) = \frac{1}{1 + e^{-v_{ij}}}$

粒子的位置只有(0,1)两种状态。速度值越大，则粒子位置取1的可能性越大，反之越小。

为了防止 $\text{sigmoid}(v)$ 饱和，Kennedy等建议将 v_{ij} 限制在 $[-4,4]$ 之间。

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

基本PSO更新公式

- ◆ 位置和速度更新公式
- ◆ 多种群PSO
- ◆ 种群拓扑结构
- ◆ 与其他智能优化算法的混合
- ◆ 拓展PSO算法用于：

多目标优化

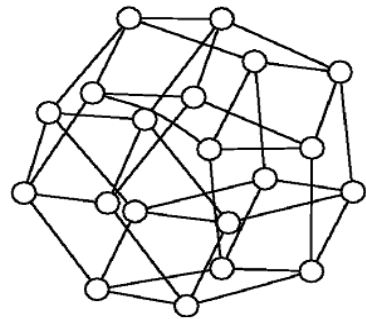
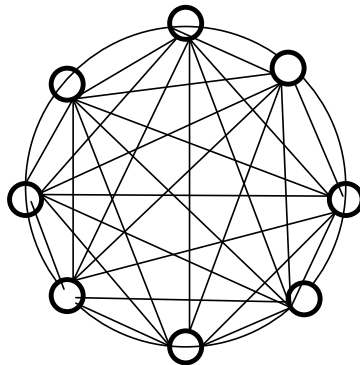
约束优化

离散优化

动态优化

python中的粒子群算法库、包：
pyPSO、scikit-opt、deap

全局PSO



Von Neumann 拓扑
(Mendes和Kennedy, 2002)

局部PSO

